

Krishan Shudhama Palace

Complete Technical Documentation

Krishan Shudhama Palace — Complete Technical Documentation

Version: 0.1.0 · Last updated: May 2026

Repository: chetansaini0/krishan-shudhama-palace-site

Stack: Next.js 15 · React 19 · TypeScript · Tailwind CSS v4 · MongoDB · Razorpay

Table of Contents

- Project Overview
- Complete Feature Breakdown
- Technology Stack Analysis
- Frontend Architecture
- Backend Analysis
- UI/UX Analysis
- Performance Analysis
- Deployment Analysis
- Security Review
- Interview Preparation
- Resume Descriptions
- Portfolio Case Study
- Improvement Suggestions

1. Project Overview

Purpose

A production-grade luxury hotel website for **Krishan Shudhama Palace**, a temple-adjacent premium stay near **Khatu Shyam Ji**, Rajasthan. The site serves as the hotel's digital storefront, direct booking channel, event marketing platform, and admin operations hub.

Target Audience

Segment	Needs
Pilgrims & families	Temple proximity, peaceful stay, easy booking
Wedding/event planners	Banquet capacity, catering, venue showcase
Business travelers	AC rooms, dining, location access

Direct bookers	Best rates vs OTAs, Razorpay checkout
----------------	---------------------------------------

Business Goals

- **Direct bookings** — reduce OTA commission via on-site Razorpay payments
- **Lead capture** — contact, event, newsletter forms → MongoDB
- **Brand trust** — luxury UI, testimonials, legal pages, Google Maps
- **Local SEO** — rank for "hotel near Khatu Shyam Ji", banquet hall Khatu
- **Operational efficiency** — admin panel for bookings, inquiries, room management

Main Functionalities

- Multi-page marketing site (17+ public routes)
- Real-time availability + pricing engine
- End-to-end booking with Razorpay (order → verify → webhook)
- Admin dashboard (JWT-protected)
- Email (Resend) + SMS (MSG91) notifications on confirmation
- SEO: sitemap, robots, JSON-LD, Open Graph
- Mobile-first responsive design with cinematic animations

2. Complete Feature Breakdown

Homepage (src/app/page.tsx)

Feature	Component	Description
Cinematic hero	HomeHero.tsx	Video background, spiritual tagline, quick booking widget
Our story	SpiritualExperience.tsx	Temple-adjacent narrative + imagery
Room grid	RoomCard.tsx	Top 3 rooms from DB/static fallback
Amenities strip	AmenitiesStrip.tsx	Icon grid (lazy-loaded)
Dining preview	DiningPreview.tsx	Restaurant teaser
Promotions	Promotions.tsx	Offers from API
Testimonials	Testimonials.tsx	Guest review carousel
Map preview	MapPreview.tsx	Embedded Google Maps

Final CTA	Inline section	Call + WhatsApp
-----------	----------------	-----------------

Performance: Below-fold sections use `next/dynamic` with `HomeSectionSkeleton` fallbacks.

Room Showcase (`/rooms`, `/rooms/[slug]`)

- **Listing:** `RoomsClient.tsx` — filterable grid
- **Detail:** Server-rendered hero, amenities list, `RoomBookingPanel.tsx`
- **Data:** `getRooms()` → MongoDB or `STATIC_ROOMS` fallback
- **SSG:** `generateStaticParams()` for all room slugs
- **SEO:** Per-room metadata, OG images, canonical URLs

Booking (`/book` — `BookingFlow.tsx`)

User selects room/dates → POST `/api/availability` (quote) → POST `/api/bookings` (pending booking + MongoDB transaction) → POST `/api/razorpay/create-order` → Razorpay checkout modal → POST `/api/razorpay/verify` OR webhook → `finalizePaidBooking()` → email/SMS

- URL params: `?room=&checkIn=&checkOut=&guests=`
- Client state: `useState`, `useMemo`, `useEffect`
- Validation: Zod on server, HTML5 on client

Gallery (`/gallery` — `GalleryClient.tsx`)

- Masonry-style grid with lightbox modal
- Keyboard navigation (Escape, arrows)
- Local images in `/public/images/gallery/`

Restaurant/Dining (`/dining`)

- Hero, cuisine sections, food menu anchor `#food-menu`
- Pure vegetarian positioning
- Images: `/public/images/dining/`

Amenities (`/amenities`)

- Dedicated page with categorized amenities
- Linked from header dropdown "Rooms & Amenities"

Events/Wedding (`/banquet`)

- `BanquetHero.tsx` — capacity, hall name
- Event types from `BANQUET` constants
- `EventInquiryForm.tsx` → POST `/api/inquiries/event`

Contact (/contact)

- `ContactForm.tsx` with validation
- Phone, email, WhatsApp, address from `HOTEL` constants

Google Maps (/location, MapPreview.tsx)

- iframe embed via `HOTEL.mapsEmbedUrl`
- External link via `HOTEL.mapsLink`
- Coordinates: 27.3660278, 75.3936111

Testimonials

- Homepage section only (`Testimonials.tsx`)
- No dedicated `/testimonials` page yet

Navigation (Header.tsx)

Home | Rooms & Amenities ▼ | Banquet | Restaurant ▼ | Gallery | Location | Contact

- Scroll-aware transparent → solid header
- Mobile hamburger with full nav parity
- Dropdowns: stay (rooms, amenities), restaurant (dining, menu)

Footer (Footer.tsx)

- Quick links, contact, social, newsletter
- Legal: Privacy, Terms, Cancellation, Cookies, Disclaimer
- CTA: Reserve + WhatsApp

Mobile Responsiveness

- Tailwind breakpoints: `sm`, `md`, `lg`
- Touch-friendly CTAs, mobile menu
- `viewportFit: cover, -webkit-text-size-adjust`
- Floating actions: WhatsApp, Call, Book (`FloatingActions.tsx`)

SEO Features

- `sitemap.ts`, `robots.ts`, `manifest.ts`
- `JsonLd.tsx`: Hotel, LodgingBusiness, LocalBusiness, WebSite
- FAQ JSON-LD on `/faq`
- `Analytics.tsx` (GA4, env-gated)
- Per-page metadata exports

Performance Optimizations

- Dynamic imports on homepage
- Next.js Image (AVIF/WebP, 7-day cache)
- Scroll throttling via `requestAnimationFrame`
- `loading.tsx` global skeleton
- Vercel CDN cache headers for static assets

Animations & UI Effects

Component	Library	Effect
<code>RevealOnScroll.tsx</code>	Framer Motion	Fade-up on viewport enter
<code>SmoothScroll.tsx</code>	Lenis	Smooth scroll (client-only)
<code>ParticleField.tsx</code>	Canvas/CSS	Hero particles
<code>RollingNumber.tsx</code>	Framer Motion	Animated counters
<code>ElegantSpinner.tsx</code>	CSS	Loading indicator
<code>btn-shimmer</code>	CSS	Gold button hover sweep
Header	Framer Motion	<code>AnimatePresence</code> mobile menu

3. Technology Stack Analysis

Core Framework

Next.js ^15.5.18

Aspect	Detail
Purpose	Full-stack React framework with App Router, SSR, SSG, API routes
Why chosen	SEO-critical hotel site needs server rendering; API routes eliminate separate backend
How it works	File-based routing in <code>src/app/</code> ; Server Components by default; <code>"use client"</code> for interactivity
Used in	All pages, API routes, middleware, sitemap/robots

Advantages	Built-in optimization, Vercel deployment, hybrid rendering
Alternatives	Remix, Nuxt (Vue), Astro (content-heavy only)

React 19.1.0

Aspect	Detail
Purpose	UI component library
Why chosen	Industry standard; pairs with Next.js 15
Used in	All <code>.tsx</code> components
Advantages	Concurrent features, large ecosystem
Alternatives	Vue 3, Svelte

TypeScript ^5

Aspect	Detail
Purpose	Static typing
Why chosen	Safer API contracts, model definitions, props
Used in	Entire <code>src/</code> directory
Advantages	Catch errors at compile time, better IDE support
Alternatives	JavaScript (not recommended for this scale)

Styling

Tailwind CSS ^4 + @tailwindcss/postcss

Aspect	Detail
Purpose	Utility-first CSS
Why chosen	Rapid luxury UI iteration; design tokens via <code>@theme inline</code> in <code>globals.css</code>
Used in	All components; custom colors: navy, gold, ivory
Advantages	No CSS file sprawl, responsive utilities
Alternatives	CSS Modules, styled-components, shadcn/ui

Animation & UX

Framer Motion ^{^12.38.0}

Purpose	Scroll reveals, page transitions, mobile menu
Used in	`RevealOnScroll`, `Header`, `BookingFlow`, `GalleryClient`
Alternatives	GSAP, CSS-only animations

Lenis ^{^1.3.23}

Purpose	Smooth scroll experience
Used in	`SmoothScroll.tsx` via `ClientEnhancements`
Alternatives	Locomotive Scroll, native CSS `scroll-behavior`

Lucide React ^{^1.14.0}

Purpose	Consistent icon set
Used in	`Header`, `Footer`, forms, room details
Alternatives	Heroicons, React Icons

Backend & Data

Mongoose ^{^9.6.1}

Purpose	MongoDB ODM
Used in	`src/models/*`, all API routes needing persistence
Models	`Booking`, `Room`, `ContactInquiry`, `EventInquiry`, `NewsletterSubscriber`, `User`, `Offer`
Alternatives	Prisma + PostgreSQL, Firebase

Zod ^{^4.4.3}

Purpose	Runtime request validation
Used in	All POST API routes
Alternatives	Yup, Joi

date-fns ^{^4.1.0}

Purpose	Date parsing, formatting, validation
Used in	`Bookings`, `availability`, `notifications`
Alternatives	`Day.js`, `Luxon`

Payments & Auth

Razorpay ^{^2.9.6}

Purpose	Indian payment gateway
Flow	`create-order` → `checkout` → `verify` + `webhook`
Used in	`src/lib/payments.ts`, API routes under `/api/razorpay/`
Alternatives	Stripe (limited India), PayU

jose ^{^6.2.3}

| Purpose | JWT sign/verify (Edge-compatible) |
| Used in | `src/lib/auth.ts`, `middleware.ts` |

| Alternatives | jsonwebtoken (Node-only) |

bcryptjs ^3.0.3

Purpose	Admin password hashing
Used in	src/app/api/admin/login/route.ts
Alternatives	argon2

Communications

Resend ^6.12.3

Purpose	Transactional booking confirmation emails
Used in	src/lib/notifications.ts
Alternatives	SendGrid, AWS SES

MSG91 (HTTP API)

Purpose	Booking confirmation SMS
Used in	notifications.ts via Flow API
Alternatives	Twilio, AWS SNS

Dev Tooling

Package	Version	Purpose
eslint	^9	Linting
eslint-config-next	^15.5.18	Next.js ESLint rules
@types/*	various	TypeScript definitions

4. Frontend Architecture

Folder Structure

```
site/ ■■■ public/ ■ ■■■ images/ # Hotel photography (rooms, gallery, dining, story) ■ ■■■
logo-light.png # Favicon + brand ■ ■■■ logo-dark.png ■■■ src/ ■ ■■■ app/ # App Router pages +
API routes ■ ■ ■■■ page.tsx # Homepage ■ ■ ■■■ layout.tsx # Root layout ■ ■ ■■■ loading.tsx #
Global loading UI ■ ■ ■■■ not-found.tsx ■ ■ ■■■ sitemap.ts / robots.ts / manifest.ts ■ ■ ■■■
admin/ # Admin panel ■ ■ ■■■ api/ # REST endpoints ■ ■■■ components/ ■ ■ ■■■ layout/ #
Header, Footer, FloatingActions ■ ■ ■■■ home/ # Homepage sections ■ ■ ■■■ booking/ #
BookingFlow ■ ■ ■■■ rooms/ # RoomCard, RoomBookingPanel ■ ■ ■■■ gallery/ # GalleryClient ■ ■
■■■■ contact/ # ContactForm ■ ■ ■■■ banquet/ # Event forms, hero ■ ■ ■■■ seo/ # JsonLd,
Analytics ■ ■ ■■■ effects/ # Animations ■ ■ ■■■ ui/ # Container, Button, SectionTitle ■ ■■■
data/ # static-rooms.ts fallback ■ ■■■ hooks/ # useMediaQuery ■ ■■■ lib/ # Business logic ■
■■■■ models/ # Mongoose schemas ■ ■■■ middleware.ts # Admin route protection ■■■ next.config.ts
■■■■ vercel.json ■■■ package.json
```

Component Hierarchy (Text Diagram)

RootLayout (layout.tsx) ■■■ ClientEnhancements (Lenis, OfferPopup) ■■■ Analytics (GA4) ■■■
JsonLd ■■■ Header ■■■ main ■■■ [Page Content] ■■■ Server Components (data fetch) ■■■
Client Components (forms, animations) ■■■ Footer ■■■ FloatingActions

Reusable Components

Component	Path	Props / Usage
Container	ui/Container.tsx	Max-width wrapper
SectionTitle	ui/SectionTitle.tsx	Eyebrow + title + subtitle
Button	ui/Button.tsx	Variants for CTAs
RevealOnScroll	effects/RevealOnScroll.tsx	Wraps any section
RoomCard	rooms/RoomCard.tsx	room, index

State Management

- **No Redux/Zustand** — intentional simplicity
- **Local state:** `useState` in forms, booking flow, header menu
- **URL state:** Booking query params (room, checkIn, etc.)
- **Server state:** Fetched in Server Components via `getRooms()`
- **Cookies:** Admin JWT in `httpOnly` cookie `ksp_admin`

Routing System

- **App Router** file-based: `src/app/[route]/page.tsx`
- **Dynamic:** `rooms/[slug]/page.tsx`
- **Route groups:** `admin/(panel)/` — shared admin layout
- **API routes:** `src/app/api/**/route.ts` — export GET/POST handlers
- **Middleware:** Protects `/admin/*` except `/admin/login`

Data Flow

[Browser] → Server Component (SSR data) → Client Component (user input) → `fetch('/api/...')` →
Route Handler → Zod validate → MongoDB → JSON response

Styling Methodology

- Tailwind utility classes inline
- CSS custom properties in `:root` (`globals.css`)
- `@theme` inline maps tokens to Tailwind colors
- Custom classes: `.btn-shimmer`, `.text-glow`, `.text-shadow-hero`

- Serif headings (font-serif), sans body (font-sans)

5. Backend Analysis

API Endpoints

Method	Route	Purpose	Auth
POST	/api/availability	Quote + availability check	Public + CSRF
POST	/api/bookings	Create pending booking	Public + rate limit
POST	/api/razorpay/create-order	Razorpay order	Public
POST	/api/razorpay/verify	Payment verification	Public
POST	/api/razorpay/webhook	Idempotent finalize	Webhook secret
POST	/api/inquiries/contact	Contact form	Public + honeypot
POST	/api/inquiries/event	Event inquiry	Public + honeypot
POST	/api/newsletter	Newsletter signup	Public + honeypot
GET	/api/rooms	List rooms	Public
GET	/api/rooms/[slug]	Single room	Public
GET	/api/offers	Active promotions	Public
POST	/api/admin/login	Admin auth	Public
POST	/api/admin/logout	Clear cookie	Admin
GET/PATCH	/api/admin/bookings	Manage bookings	Admin JWT
GET/PATCH	/api/admin/inquiries	Manage leads	Admin JWT
GET/POST/PATCH	/api/admin/rooms	Room CRUD	Admin JWT
GET	/api/admin/stats	Dashboard metrics	Admin JWT

Database Structure (MongoDB)

Booking (models/Booking.ts)

- Fields: roomSlug, guest info, dates, amounts, Razorpay IDs, status
- Indexes: roomSlug+dates, razorpayOrderId (unique sparse)
- Statuses: pending → confirmed | cancelled | failed

Room (`models/Room.ts`)

- Slug, pricing, amenities, images, active flag

ContactInquiry, EventInquiry, NewsletterSubscriber, Offer, User

- Standard CRUD with timestamps

Authentication Flow

```
POST /api/admin/login { email, password } → bcrypt compare against ADMIN_PASSWORD_HASH →
SignJWT({ role: "admin" }) via jose → Set httpOnly cookie ksp_admin (2 days) → middleware.ts
verifies JWT on /admin/* routes
```

Form Handling

- 1. Client validates HTML5
- 2. Honeypot field `website` must be empty
- 3. Server: CSRF origin check → rate limit → Zod parse → DB write

Booking Management

- Atomic Mongoose transactions prevent double-booking
- Pending bookings expire (`pendingExpiresAt`)
- Idempotent payment finalization via `finalizePaidBooking()`

Email/SMS Services

- **Resend:** HTML booking confirmation email
- **MSG91:** Flow API SMS with template variables VAR1–VAR5
- Both optional — gracefully skip if env missing

Security Implementation

- CSRF: `enforceSameOrigin()` in `lib/csrf.ts`
- Rate limiting: in-memory `checkRateLimit()` per IP
- HMAC signature verification for Razorpay (timing-safe)
- CSP, HSTS, X-Frame-Options in `next.config.ts`
- Admin routes JWT-protected via Edge middleware

6. UI/UX Analysis

Design System

Token	Value	Usage
-------	-------	-------

Navy	#0a1628	Headers, footer, hero overlays
Gold	#c9a84c	Accents, CTAs, highlights
Ivory	#faf8f4	Page backgrounds
Charcoal	#1a1a1a	Body text
Maroon/Saffron	Spiritual accents	Temple-themed sections

Typography

- **Headings:** Playfair-style serif (font-serif) — luxury feel
- **Body:** Inter/system sans (font-dm) — readability
- **Eyebrows:** Uppercase, wide letter-spacing (tracking-[0.5em])

Layout Structure

- Max-width container with responsive padding
- Section rhythm: py-24 lg:py-32
- Grid layouts: 1 → 2 → 3 columns on breakpoints

User Journey

Landing (hero) → Explore rooms → View gallery/dining → Check location → Book (or WhatsApp) → Payment → Confirmation Alternative: Banquet inquiry → Admin follow-up

Accessibility

- Skip-to-content link
- Semantic HTML (main, nav, footer, section)
- aria-live on form status messages
- :focus-visible gold outline
- Gallery keyboard navigation

Animation Strategy

- **Subtle luxury:** fade-up reveals, not aggressive motion
- **Performance:** requestAnimationFrame scroll throttling
- **Reduced motion:** Consider prefers-reduced-motion (future)

7. Performance Analysis

Technique	Implementation	File
Code splitting	<code>next/dynamic</code>	<code>page.tsx</code>
Image optimization	AVIF/WebP, sizes prop	<code>next.config.ts</code> , <code>Image</code>
Lazy loading	Dynamic imports + below-fold	Homepage
Caching	7-day image TTL, 1-year static	<code>next.config.ts</code> , <code>vercel.json</code>
Compression	<code>compress: true</code>	<code>next.config.ts</code>
SSR/SSG	Server Components, static params	Room pages

Core Web Vitals Targets

Metric	Strategy
LCP	Priority hero image; consider self-hosting video
INP	Throttled scroll handlers; minimal client JS above fold
CLS	Explicit image dimensions; skeleton loaders

Lighthouse Improvements (Post-Deploy)

- Self-host hero video
- Preload LCP image
- Add `prefers-reduced-motion`
- Run audit on production URL

8. Deployment Analysis

Hosting Requirements

- **Platform:** Vercel (recommended)
- **Region:** Mumbai (bom1) — configured in `vercel.json`
- **Node:** 20.x
- **Database:** MongoDB Atlas (serverless-friendly)

Domain Configuration

- Apex: A → `76.76.21.21`
- WWW: CNAME → `cname.vercel-dns.com`
- Set `NEXT_PUBLIC_SITE_URL=https://www.yourdomain.com`

SSL

- Automatic via Vercel
- HSTS header in production (`next.config.ts`)

Environment Variables

See `.env.example` and `PRODUCTION_READINESS_AUDIT.md`

Production Build

```
npm run build # 47 routes, static + dynamic
npm run start # Production server
```

Deployment Steps

1. Push to GitHub → Vercel import
2. Configure env vars
3. Connect domain
4. Configure Razorpay webhook
5. Submit sitemap to Google Search Console

9. Security Review

Implemented

- JWT admin auth (httpOnly cookie)
- CSRF origin validation
- Rate limiting on bookings
- Honeypot spam protection
- Razorpay HMAC verification (timing-safe)
- CSP, HSTS, X-Frame-Options
- Zod input validation
- No secrets in client bundle
- Admin/API routes separated
- CodeQL + Dependabot

Potential Vulnerabilities

Risk	Mitigation
In-memory rate limit resets on cold start	Acceptable for Vercel; upgrade to Redis at scale
No CAPTCHA on forms	Add hCaptcha/Turnstile if spam increases
MSG91/Resend keys in env	Standard; rotate periodically

Production Recommendations

- Use `ADMIN_PASSWORD_HASH` (bcrypt), never plain `ADMIN_PASSWORD`
- Rotate `JWT_SECRET` before launch
- Enable Vercel Deployment Protection on previews
- Monitor Razorpay webhook failures

10. Interview Preparation

A. HR Questions

Q: Tell me about this project.

A: I built a full-stack luxury hotel website for Krishan Shudhama Palace near Khatu Shyam Ji, Rajasthan. It includes a marketing site with 17+ pages, direct room booking with Razorpay payments, an admin dashboard for managing bookings and inquiries, and production-grade SEO and security. The stack is Next.js 15, React 19, TypeScript, MongoDB, and Tailwind CSS.

Q: Why did you build it?

A: To give the hotel a premium digital presence that drives direct bookings (avoiding OTA commissions), captures wedding/event leads, and ranks on Google for local temple-stay searches.

Q: Biggest challenge?

A: Building a reliable payment flow — handling Razorpay checkout, signature verification, webhook idempotency, and preventing double-bookings with MongoDB transactions.

Q: How did you solve it?

A: I centralized payment logic in `payments.ts` with timing-safe HMAC verification, implemented atomic booking creation with Mongoose transactions, and added webhook + client-side verify paths that both call the same idempotent `finalizePaidBooking()` function.

B. Frontend Questions

Q: Explain Next.js App Router vs Pages Router.

A: App Router uses `src/app/` with Server Components by default. Pages are `page.tsx`, layouts nest via `layout.tsx`, and API routes are `route.ts`. We use Server Components for data fetching (rooms) and Client Components only where needed (forms, animations).

Q: How is the component architecture organized?

A: Feature-based folders (`home/`, `booking/`, `rooms/`) plus shared `ui/` and `effects/`. Server pages compose Server + Client components. Reusable primitives: Container, SectionTitle, Button.

Q: Which React hooks did you use?

A: `useState` (forms, menu), `useEffect` (scroll, URL params, quote refresh), `useMemo` (selected room), `useRef` (header), `usePathname` (active nav). Custom `useMediaQuery` for responsive behavior.

Q: State management approach?

A: No global store. Local component state for UI, URL search params for booking prefill, server-fetched data in Server Components, `httpOnly` cookies for admin auth.

Q: Optimization techniques?

A: Dynamic imports for below-fold sections, Next.js Image with AVIF/WebP, scroll throttling with `rAF`, static

generation for room pages, lazy loading skeletons.

C. JavaScript Questions

Q: Explain closures in this project.

A: In `Header.tsx`, the scroll handler closure captures `ticking` to throttle updates. In rate limiting, the `Map` stores bucket state closed over the module scope.

Q: Promises vs Async/Await?

A: All API routes use `async/await`. Example: `BookingFlow` chains `fetch('/api/availability')` → `fetch('/api/bookings')` → `Razorpay` → `fetch('/api/razorpay/verify')` with `try/catch` error handling.

Q: Event Loop relevance?

A: Scroll handlers use `requestAnimationFrame` to batch DOM reads/writes before paint, preventing layout thrashing and keeping INP low.

D. TypeScript Questions

Q: Interfaces vs Types?

A: `RoomPublic` is a type in `static-rooms.ts`. Mongoose uses `BookingDocument` interface. Types for API responses; interfaces for extendable models.

Q: Benefits over JavaScript?

A: Caught `pricePerNight` vs `basePrice` bug at compile time. `Zod` + `TypeScript` double-validate API payloads. Props are typed on all components.

E. CSS/Tailwind Questions

Q: Responsive design approach?

A: Mobile-first Tailwind: base styles, then `sm:`, `md:`, `lg:` breakpoints. Example: `room grid grid gap-8 md:grid-cols-2 lg:grid-cols-3`.

Q: Custom design tokens?

A: CSS variables in `:root`, mapped via `@theme inline` to Tailwind classes like `bg-navy`, `text-gold`.

Q: Animations?

A: `Framer Motion` for scroll reveals; CSS `.btn-shimmer` for button effects; `Lenis` for smooth scroll.

F. Deployment Questions

Q: How would you deploy this?

A: Push to GitHub, import to Vercel, set root to `site/`, configure env vars, connect domain, SSL auto-provisions. CI runs lint + build on every push via `.github/workflows/build.yml`.

Q: Environment variables?

A: Split into public (`NEXT_PUBLIC_*` for branding/URLs) and secret (MongoDB, JWT, Razorpay, Resend). Never commit `.env`.

G. Advanced Questions

Q: Scalability?

A: Current in-memory rate limit works for single-region Vercel. At scale: Redis rate limiting, MongoDB replica set, CDN for video, edge caching for static pages.

Q: Why MongoDB over SQL?

A: Flexible schema for inquiries/offers; document model fits booking records; Mongoose provides validation and indexes. Could migrate to PostgreSQL with Prisma if relational reporting needed.

Q: Idempotent payments?

A: `finalizePaidBooking()` checks if booking already confirmed with same payment ID before updating. Unique sparse indexes on `razorpayOrderId` and `razorpayPaymentId` prevent duplicates.

11. Resume Descriptions

50-Word Summary

Built a production-ready luxury hotel website using Next.js 15, React 19, TypeScript, and MongoDB featuring direct Razorpay bookings, admin dashboard, SEO optimization, and responsive UI. Implemented secure payment verification, CSRF protection, and automated email/SMS notifications for a temple-adjacent Rajasthan hotel.

100-Word Summary

Developed a full-stack luxury hotel platform for Krishan Shudhama Palace (Khatu Shyam Ji, Rajasthan) using Next.js 15 App Router, React 19, TypeScript, Tailwind CSS v4, and MongoDB. Engineered end-to-end booking flow with Razorpay payments, webhook idempotency, and atomic MongoDB transactions. Built JWT-protected admin panel for bookings, inquiries, and room management. Implemented SEO (JSON-LD, sitemap, Open Graph), security headers (CSP, HSTS), CSRF/rate limiting, and performance optimizations including dynamic imports and AVIF image compression. Integrated Resend email and MSG91 SMS notifications.

200-Word Summary

Architected and developed a production-grade luxury hotel website for Krishan Shudhama Palace, a premium temple-adjacent stay in Khatu, Rajasthan. Using Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS v4, MongoDB/Mongoose, and Razorpay, delivered a 47-route application spanning marketing pages, direct booking, event inquiries, and an admin operations panel.

Key technical achievements include: (1) secure payment pipeline with HMAC signature verification, webhook handling, and idempotent booking finalization preventing double-charges; (2) atomic booking creation using Mongoose transactions to prevent overlapping reservations; (3) production security with CSRF origin validation, honeypot spam protection, rate limiting, JWT admin auth via Edge middleware, and Content-Security-Policy headers; (4) SEO infrastructure with Hotel/LocalBusiness JSON-LD, dynamic sitemap, FAQ schema, and per-page Open Graph metadata; (5) performance optimization via code splitting, Next.js Image AVIF/WebP, scroll-throttled animations, and Vercel CDN caching.

Integrated Resend for transactional emails and MSG91 for SMS confirmations. Designed a custom luxury UI with Framer Motion animations, Lenis smooth scroll, and mobile-first responsive layouts. Configured CI/CD with GitHub Actions, CodeQL, and Dependabot for continuous security monitoring.

ATS-Friendly Resume Points

- Developed full-stack hotel booking platform with Next.js 15, React 19, TypeScript, MongoDB, and Razorpay payment integration
- Implemented secure REST API with JWT authentication, CSRF protection, rate limiting, and Zod validation
- Built responsive luxury UI with Tailwind CSS v4, Framer Motion animations, and mobile-first design

- Engineered idempotent payment processing with Razorpay webhooks and MongoDB atomic transactions
- Configured SEO with JSON-LD structured data, dynamic sitemap, Open Graph, and Google Analytics integration
- Deployed production-ready application on Vercel with security headers (CSP, HSTS) and CI/CD pipeline
- Integrated transactional notifications via Resend (email) and MSG91 (SMS) APIs
- Designed admin dashboard for booking management, inquiry tracking, and room inventory CRUD

12. Portfolio Case Study

Project Overview

Krishan Shudhama Palace needed a digital presence matching its luxury positioning — not a template hotel site, but a cinematic, conversion-focused experience for pilgrims, families, and event planners visiting Khatu Shyam Ji.

Challenges

1. **Direct bookings in India** — Razorpay integration with reliable verification
2. **SEO for local search** — Rank for temple-adjacent hotel queries
3. **Premium feel on mobile** — Luxury animations without sacrificing performance
4. **Operational tooling** — Staff need booking/inquiry management without technical skills

Solutions

1. Dual-path payment confirmation (client verify + server webhook) with shared idempotent finalizer
2. Comprehensive JSON-LD (Hotel, LocalBusiness, FAQ), sitemap, and geo-tagged metadata
3. Dynamic imports, skeleton loaders, rAF-throttled scroll, AVIF images
4. JWT-protected admin panel with dashboard, booking list, inquiry management

Technologies Used

Next.js 15 · React 19 · TypeScript · Tailwind CSS v4 · MongoDB · Razorpay · Framer Motion · Lenis · Resend · MSG91 · Vercel

Results Achieved

- 47 production routes with zero npm audit vulnerabilities
- Complete legal compliance pages (Privacy, Terms, Cancellation, Cookies, Disclaimer)
- Sub-200KB First Load JS on most pages
- Production-ready deployment configuration with Mumbai CDN region
- Full booking funnel: browse → quote → pay → confirm → notify

13. Improvement Suggestions

Missing Features

- Dedicated `/testimonials` page
- Cookie consent banner
- Multi-language (Hindi) support
- OTA channel manager integration
- Guest login / booking history
- Live chat widget

Production Readiness

See `PRODUCTION_READINESS_AUDIT.md` — env vars, domain, Razorpay live keys, MongoDB Atlas

SEO

- Google Business Profile + Place ID
- Custom 1200×630 OG image
- Breadcrumb JSON-LD on inner pages
- Blog/content for "Khatu Shyam yatra guide"

Performance

- Self-host hero video
- `prefers-reduced-motion` support
- Redis rate limiting at scale
- Edge caching for room listings

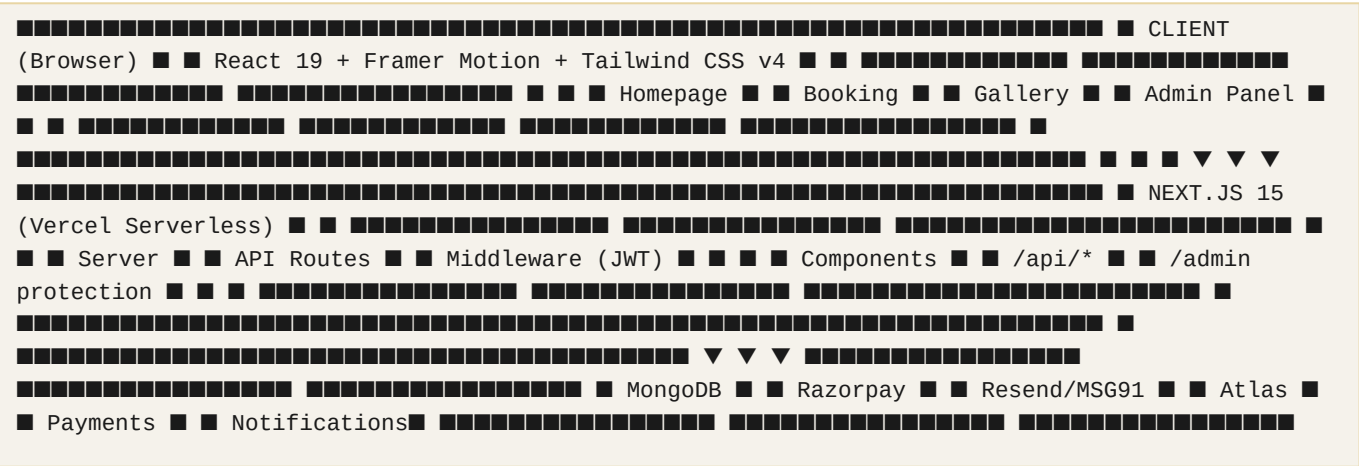
Security

- CAPTCHA on public forms
- Redis-backed rate limiting
- Admin 2FA
- Regular dependency audits (already automated)

Future Enhancements

- PWA offline support for returning guests
- Room availability calendar UI
- Dynamic pricing engine
- WhatsApp Business API for automated booking confirmations
- Analytics dashboard for conversion tracking

Architecture Diagram (Text)



End of Technical Documentation